

LAB ASSIGNMENT Nº1

WiFi-BASED ULTRASONIC DETECTOR

(FIREBASE, ANDROID APP AND ESP32-C6-DEVKITC-1)

1 INTRODUCTION

This assignment is intended to be followed by the students in order to give their first steps in Android IoT application programming. In the course of this tutorial, a simple Android app will be developed and connected to a Google Firebase project. The developed app will be used as a basis for subsequent lab assignments. It is assumed that the students have the Android Studio IDE installed in their machines.

The instructions in this assignment are abridged so that, at each step, the students will have to investigate how to implement the requested functionality. At the end of this document, a list of useful references is provided.

2 CREATING A GOOGLE FIREBASE PROJECT

This sequence steps will guide you through the creation of a Google Firebase project that will store the data of your IoT application in the cloud.

- 1) Sign-in to Google or register a Google account.
- 2) Go to [Google Firebase](#).
- 3) Go to the console.
- 4) Create a project called “IoT Alarm App” with default settings and location in Portugal.
- 5) Create a Realtime Database database. Realtime Database is used instead of Cloud Firestore because it is compatible with existing Arduino libraries that will be used with Arduino devices in later assignments. Start the database in **locked mode** for restricted access.
- 6) Go to the Data separator of the just created **iot-alarm-app** database and add the following items:
 - a. *alarm_status*, setting it to Boolean value **true**.
 - b. *enable_alert_sound*, setting it to **true**.
 - c. *measurement*, setting it to 0.
- 7) Go to database Rules separator and check that **.read** and **.write** rules are both set to **true**.
- 8) In the Firebase menu, Go to *Build/Authentication/Sign-in method*. Enable the *Anonymous* method.

3 CREATING A SIMPLE APP

This sequence of steps will guide you through the creation of a basic app layout, which will later be completed with functionality allowing interaction between the user and the Google Firebase database.

- 1) Create a new project called “Assignment 1 GR00”, with project type “Empty Views Activity” and minimum SDK version 21 (Android 5.0 Lollipop). The *GR00* indicates the RMIC lab group number, in this case group 0. The language should be “**Java**”.
- 2) Verify that the new project has a single activity called *MainActivity*, whose class is defined in file *MainActivity.java*. The layout of *MainActivity* is very simple and has only two widgets: a **ConstraintLayout** with a **TextView** inside.
- 3) Modify the layout so that it will include two **TextView** gadgets and one **Button** gadget positioned as shown in Fig. 1. The default text in each gadget indicates the database item to which it will be associated. You can check the layout using the [Android Emulator](#).
- 4) **Customize the app icon to be the IST logo!**

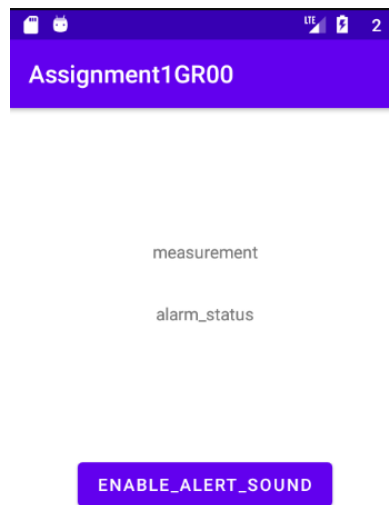


Fig. 1: Default layout of app Assignment 1.

4 CONNECT THE ANDROID APP TO THE GOOGLE FIREBASE PROJECT

In Android Studio, select the menu option *Tools/Firebase*. Then, select *Realtime Database*, and then *Get started with Realtime Database [Java]*. Follow the steps.

At the end of this sequence of steps, the Android app will allow full interaction between the user and the Google Firebase database.

- 1) Add functionality to class **MainActivity**, so that the value of the *measurement* database item is shown in the *measurement TextView*.
- 2) Add functionality to class **MainActivity** class, so that:
 - a. Whenever the *alarm_status* database item is set to **false** the *alarm_status TextView* displays “Nothing to Report”.

¹ <https://developer.android.com/studio/write/image-asset-studio#create-adaptive>

- b. Whenever the *alarm_status* database item is set to **true** the *alarm_status* **TextView** displays “ALERT! ALERT!” and one notification sound is played.
- 3) Add functionality to class **MainActivity** class, so that:
 - a. The *enable_alert_sound* **Button** is initialized with a string that corresponds to the value of the *enable_alert_sound* database item. In case this is **true**, the button will display “Disable Buzzer”; in case it is **false**, the button will display “Enable Buzzer”.
 - b. Whenever the *enable_alert_sound* **Button** displays “Disable Buzzer” and the user presses the button, database item *enable_alert_sound* will be set to **false** and the button will display “Enable Buzzer”.
 - c. Whenever the *enable_alert_sound* **Button** displays “Enable Buzzer” and the user presses the button, database item *enable_alert_sound* will be set to **true** and the button will display “Disable Buzzer”.

Tip 1: Add the following imports, which will let you call the Firebase API methods:

- `com.google.firebase.database.DataSnapshot;`
- `com.google.firebase.database.DatabaseError;`
- `com.google.firebase.database.DatabaseReference;`
- `com.google.firebase.database.FirebaseDatabase;`
- `com.google.firebase.database.ValueEventListener;`

Tip2: Class `DatabaseReference` will allow you to get references to the Firebase database resources through calls to `DatabaseReference::getInstance().getReference().child(...)`. Get and set operations on the database items can be accomplished through `DatabaseReference::getValue(...)` and `DatabaseReference::SetValue(...)` methods².

5 ADDING ESP32-C6-DEVKITC-1 TO THE SYSTEM

The interaction between the user and the cloud database is established. In order to achieve a true IoT application, the loop will now be closed to integrate the “thing”, i.e., the sensor/actuator device. In this assignment, ESP32-C6-DEVKITC-1 will be used. Detailed information about this device can be found [here](#).

The ESP32-C6-DEVKITC-1 will receive power from the USB interface and will be connected with a HC-SR04 ultrasonic sensor (see Fig. 2) and a piezo buzzer (actuator). HC-SR04 operates at 5 V, while ESP32-C6-DEVKITC-1 operates at 3.3 V. But ESP32-C6-DEVKITC-1 has a 5 V pin that can be used to feed the HC-SR04. However, in order not to damage the ESP32-C6-DEVKITC-1, a voltage divider will be used, connecting a 1 kΩ resistor to the Echo pin, and a 1 kΩ resistor to the Trigger pin. Besides, ESP32-C6-DEVKITC-1 includes an onboard LED that can be controlled by the program.

² More info can be found at <https://firebase.google.com/docs/database/android/read-and-write>.

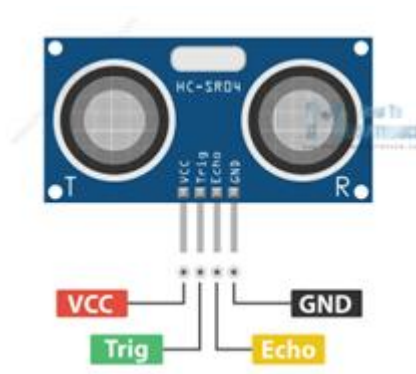


Fig. 2: HC-SR04 ultrasonic sensor.

- 1) The ESP32-C6-DEVKITC-1 will be programmed using the Arduino IDE. In order for it to be recognized in this development environment, configure Arduino IDE according to the instructions in [this link](#) or an equivalent one.
- 2) Mount the circuit establishing the connections depicted in Fig. 3. Connect the programming USB-C port of the ESP32-C6-DEVKITC-1 (the one on the same side of the RESET button, marked with the red arrow) to the PC.

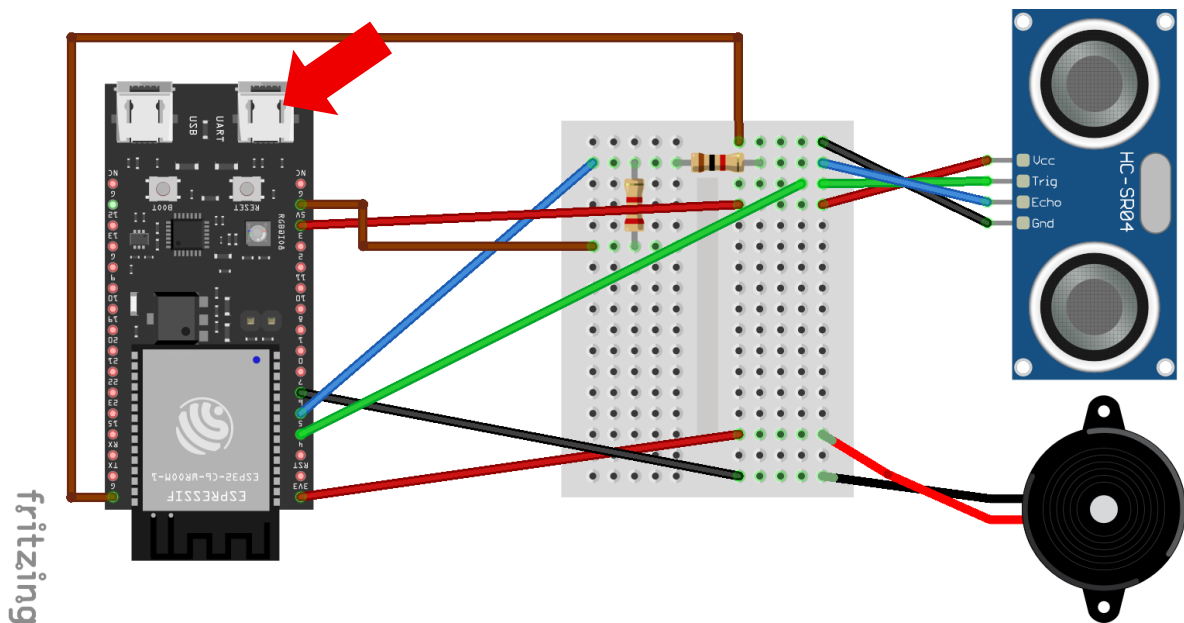


Fig. 3: Ultrasonic detector and alarm circuit using ESP32-C6-DEVKITC-1 and HC-SR04.

- 3) Install the Firebase Arduino Client Library for ESP8266 and ESP32 by Mobizt as shown in Fig. 4.

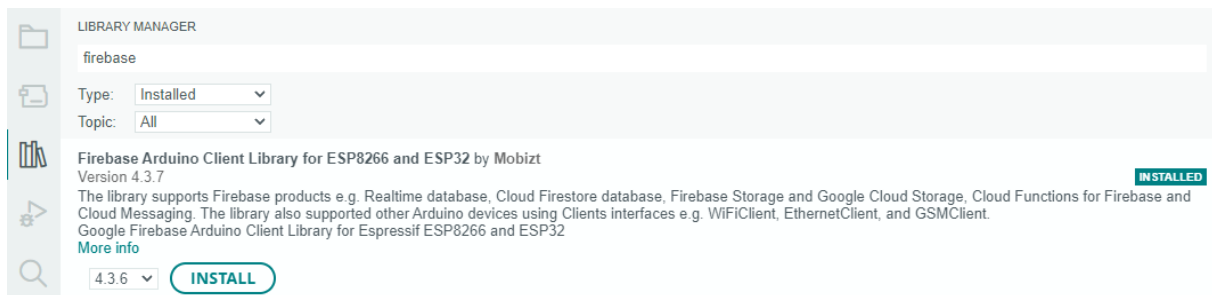


Fig. 4: Library Manager of the Arduino IDE with Firebase ESP32 client library installed.

- 4) In the Tools menu of the IDE, check that the Partition Scheme is set to **Minimal SPIFFS (1.9MB APP with OTA)**.
- 5) Expand the **lab1.zip** archive. The given **lab1.ino** program performs the following actions in the main loop (**loop()** function):
 - Reads the distance measured by the sensor. If the distance is shorter than 20 cm, it considers that an intruder is detected, otherwise, there is no alert situation. It updates the **alarm_status** database variable accordingly. In case motion is detected, it increments the **measurement** database variable and activate the onboard LED.
 - If the **enable_alert_sound** database variable is **true** and motion is detected, it sounds the buzzer (it uses the **tone()** function together with pitch constants provided in file **pitch.h**).
 - It waits for 1 second before the end of the **loop()** function.
- 6) In order to configure the **lab1.ino** file appropriately, you will need to obtain the **API_KEY** and **DATABASE_URL** parameters, which can be retrieved from the Google Firebase console. In order to obtain the API Key, go to *Project Overview/Project settings*. The Database URL can be directly retrieved from *Realtime Database*.
- 7) ESP32-C6-DevKitC-1 is based on the ESP32 chip. This chip is incompatible with *eduroam* authentication procedures. In order to connect to Internet, inside the classroom use the following WiFi network:
 - **SSID:** Labs-LSD
 - **Password:** aulaslsd
- 8) Once the ESP32-C6-DevKitC-1 program is correct, you should observe the expected hardware behavior and the Firebase database variables should change according to the motion detection events. Once the Android App is running, it should also show the current database status, as well as to enable and disable the buzzer when the user presses the button.

6 REFERENCES

- [1] Android developer guides, <https://developer.android.com/guide>.
- [2] John Horton, "Android Programming for Beginners", Packt, <https://www.packtpub.com/product/android-programming-for-beginners/9781785883262#:~:text=Android%20Programming%20for%20Beginners%20will%20be%20your%20companion,or%20are%20just%20looking%20to%20program%20for%20fun>.

- [3] Bill Phillips, Chris Stewart, Kristin, Marsicano, “Android Programming – The Big Nerd Ranch Guide”, 3rd Edition, Big Nerd Ranch, 2017.